Fault tolerance and recovery — phixeron

Contents

Fault tolerance and recovery — phixeron **1**
0. The one governing invariant: no snapshots, full-log replay . . . 1
1. Cluster / node-level fault tolerance (Java, Aeron Cluster + Raft) . . . 2
1.1 Every node holds a complete, byte-identical copy of history . . . 2
1.2 Leader failover . . . 2
1.3 A node that cannot record itself terminates (self-fencing) . . . 3
1.4 Cluster shutdown / restart . . . 3
2. FIX gateway (C++) fault tolerance . . . 3
2.1 Fencing: `closeSessions` . . . 4
2.2 Standby promotion . . . 4
2.3 Recovering FIX session state after a restart . . . 5
2.4 The accept gate and connect-once semantics . . . 6
3. Stream recovery — cold start, gaps, and the live/history split . . . 6
3.1 `ReplayerService` (Java, one per node) . . . 6
3.2 Client-side replay (`ReplayerStreamReceiver`, C++) . . . 6
4. Exactly-once query replies across a failover — `OutstandingQueries` . . . 7
5. Reference data (BasicData) recovery . . . 8
6. Operational tooling and verification . . . 9
7. What this does not cover . . . 9

Fault tolerance and recovery — phixeron

How this system survives node loss, leader failover, a stuck local archive, a crashed gateway, and a lost network frame — and how each surviving/restarted part gets back to a correct state. This is a description of what the code does today, not the aspirational superset in `doc/0-overview.md` … `doc/6-detailed-architecture.md` — cross-check against source before trusting specifics there.

0. The one governing invariant: no snapshots, full-log replay

Every recovery path in this document reduces to the same primitive: **replay the sequenced log from `globalSeqNo` 1**. `SequencerService.onTakeSnapshot` throws and `onStart` refuses a snapshot image (`src/main/java/org/limitless/phixeron/sequencer/SequencerService.java:295-300,544-550`); `clusterctl shutdown` uses Aeron's `ABORT` action, which takes no snapshot, never `SHUTDOWN` (`src/main/java/org/limitless/phixeron/tools/ClusterCtl.java:151-185` — `SHUTDOWN` snapshots first, which would silently break the invariant below).

This is deliberate, not an oversight: **every node publishes and records its own copy of the sequenced stream** (the "tap", `aeron:ipc` stream 205 — see §2), and a node restored from a snapshot would hold a recording that starts wherever the snapshot did, not at the beginning of the trading day. Full-log replay is what keeps every node's tap recording a *complete* copy

of history, which is what lets any node serve a cold-start or gap replay to its co-located apps without depending on a peer. The cost is that recovery time and archive size grow with uptime — bounded in practice because the log is scoped to one trading day (daily rollover), not unbounded (`doc/todo.md`, `[[project-no-snapshots]]` memory).

Everything downstream of the log — `globalSeqNo`, the gateway topology, FIX session sequence numbers, positions, reference data — is a **pure function of replaying that log**, so nothing needs its own persistence or its own recovery procedure. That single property is why the mechanisms below look structurally similar: kill something, let it replay, done.

1. Cluster / node-level fault tolerance (Java, Aeron Cluster + Raft)

1.1 Every node holds a complete, byte-identical copy of history

`SequencerService` runs on every cluster node — leader and follower alike — and each one re-publishes every sequenced frame onto its own node-local `aeron:ipc` tap (`FEEDER_CHANNEL`/`FEEDER_STREAM_ID`, `SequencerService.java:101-102`), which that node's co-located Aeron Archive records. Because every node processes the same Raft-committed log in the same order, the taps are byte-identical across nodes — there is no cross-node replication of the recording itself, no leader-only archive, and no asymmetry between a follower's copy of history and the leader's. The tap publication and its recording are created once in `onStart` and live for the whole process, continuous across leadership changes (`aeron:ipc` has no port to collide on, unlike the retired UDP global stream this replaced), so a node's recording is one continuous run spanning every leader tenure it lived through.

Consequence: **any node a client is co-located with can serve full history/gap replay**, and losing a node loses no history — its peers already hold an identical complete copy.

1.2 Leader failover

Raft election is Aeron Cluster's own mechanism; phixeron's contribution is what rides on top of it. `SequencerService.onNewLeadershipTermEvent` fires on every node on a new term and calls `applyLeadership`, which asks `Sequencer.leadershipChanged` to synthesize a `LeadershipChanged` frame — de-duplicated against the leader already on record, so a node re-observing its own term doesn't double-emit (`Sequencer.java:392-407`). This frame is sequenced and recorded exactly like any ingress message, so **every node's tap recording — not just the leader's — carries a gap-free account of every leadership change**, and a leader-only consumer (§3, §6) can derive "who is leader and since when" purely from replaying the log.

On the C++ client side, `ClusterStreamSender` (the Aeron Cluster ingress session state machine used by `FixGateway` and every other cluster client) survives a leader failover **without losing its cluster session**: a `NewLeaderEvent` swaps only the ingress `Publication` to the new leader's endpoint, re-resolved out of the event's member CSV — the cluster session id and leadership term id are updated in place, never re-created (`src/main/cpp/org/limitless/phixeron/sequencer/ClusterStreamSender.hpp:632-689`). `send()`'s retry loop pumps the egress control stream between offer attempts specifically so an in-flight `NewLeaderEvent` can land and swap the publication mid-spin — a naive `while(!offer) idle()` would deadlock, spinning on the dead leader's publication while the poll that would revive it never runs (`ClusterStreamSender.hpp:509-537`). A co-located client that originally reached the leader over cheap IPC and failed over onto UDP re-chases IPC if leadership later returns to its own member (`ClusterStreamSender.hpp:642-672`).

A leader failover is explicitly **not** session loss — the fencing logic in §3 treats it as a transparent event, not a fault.

1.3 A node that cannot record itself terminates (self-fencing)

`SequencerService.emit` is *reliable*: it spins on the tap-publication offer until it lands, because the recording is the authoritative copy of history and a dropped frame would be an unrecoverable gap (`SequencerService.java:640-688`). This can only block on genuine local-archive back-pressure — the tap's only tethered subscriber is the recording itself, app replicas are untethered — but reliable is not the same as unbounded. `TapStallPolicy` (pure, Aeron-free, unit-tested in isolation — `TapStallPolicy.java`) distinguishes an archive that is merely slow (back-pressured but its recording position keeps advancing → `CONTINUE`) from one that has stopped draining (`FATAL_NO_PROGRESS` after 30s of zero progress) or gone away entirely (`FATAL_RECORDING_GONE`). The same 1 Hz tick that drives the cluster clock also runs `checkTapRecordingAlive` (`SequencerService.java:480-486`), because a *stopped* recording doesn't back-pressure anything at all — the tap's untethered app subscribers keep it looking connected — so liveness has to be polled, not just inferred from back-pressure.

Either path calls `fatalTapFailure` → `fatalFailure`, which logs `FATAL: … terminating this node` and runs the fatal handler wired by `SequencerNode`, exiting the process with code **70** (`SequencerService.java:728-767`; documented operator-facing in `doc/ops.md` "A node that terminates itself"). No cluster callback may signal failure by throwing instead: `Image.boundedControlledPoll` has already advanced the log position past the message before a thrown exception is caught, and `AgentRunner` keeps the agent alive — a throw here would silently drop the frame and leave the node running with a hole in its own recording, exactly the failure this whole mechanism exists to prevent (class Javadoc, `SequencerService.java:66-74`). The same reasoning bounds `scheduleTick` (`SequencerService.java:517-538`): a consensus module that refuses the cluster-clock timer for 30s continuous back-pressure is wedged, not busy, and gets the same fatal treatment — otherwise every consumer's session clock silently stops advancing with no operator-visible signal.

Consequence for the cluster: the remaining members hold identical, complete recordings and keep quorum without the dead node (an election moves leadership if it held it); **two nodes down is a quorum loss**, not a repeat of the same event. Restarting the node is ordinary full-log replay from `globalSeqNo` 1 (§0), rebuilding its tap recording from scratch; if it exits 70 again immediately, the underlying storage is still broken (start-up itself is bounded — the recording must go live within 5s, `TAP_RECORDING_START_TIMEOUT_NS`, `SequencerService.java:121`).

1.4 Cluster shutdown / restart

`clusterctl shutdown` is leader-gated: on the leader it publishes an unsequenced `ClusterStopped` marker, best-effort awaits its sequenced echo (so the log's last event before a planned stop is always that marker), then calls Aeron's `ClusterTool.abort` (`ClusterCtl.java:151-185`). `ABORT` — not `SHUTDOWN` — is the only lifecycle action that terminates without taking a snapshot, and `SequencerNode` wires `ConsensusModule.Context.terminationHook` to a barrier so every node still unwinds cleanly (closing its Archive and draining the tap recording to disk) rather than being killed abruptly (`doc/clusterctl.md`). `clusterctl start` is the read-side complement: it publishes `ClusterStarted` and waits for its sequenced echo, so an operator/script can confirm the cluster is actually up (elected leader, ingress accepted) rather than merely that processes launched.

2. FIX gateway (C++) fault tolerance

`FixGateway` is a deliberately stateless proxy: authoritative FIX session state (sequence numbers, session status) lives in the cluster (§0), not in the gateway process, specifically so the gateway can crash and restart without losing anything durable. Two mechanisms carry the rest: fencing (stop serving before you're wrong) and standby promotion (someone else takes over).

2.1 Fencing: `closeSessions`

`FixGateway::closeSessions(reason)` (`src/main/cpp/org/limitless/phixeron/fix/FixGateway.cpp:668-696`) is the single choke point that stops this instance serving TCP clients: it snapshots every active FIX session's recoverable sequence state into `m_recoveredSessions`, releases every connectionId from the CompID registry, hard-closes every client socket (`::close(fd)` — no graceful Logout is sent; the fence deliberately looks to the cluster exactly like this process dying), and sets `m_gateOpen = false` — which also closes the accept gate, since `processSockets` only calls `acceptNewConnection()` while `m_gateOpen` is true. It is idempotent (no-ops if the gate is already shut).

Three independent signals trigger it (`FixGateway.cpp:296-325, 557-577`), and they are not equivalent — one of them is not a fault at all:

Signal	Where	What happens after
A `GatewayActive` names a **sibling** instance while this one was active	`sequencedEvent`, `FixGateway.cpp:557-577`	Fences, `m_activated = false`, but **keeps running** — drops to standby, keeps following the tap, and can be re-promoted later. The cluster session is untouched.
The cluster closes this gateway's ingress session (`ClusterStreamSender::isSessionLost()`)	`doWork`, `FixGateway.cpp:315-319`	Fences and **exits the process** (`running = false`) — a lost session cannot be re-established in-process (§2.3), so an instance in this state could never be promoted again; fail closed by exiting rather than idling with no session.
No `Tick` from the co-located tap for `TAP_STALL_TIMEOUT_MS` (20s = 20× the 1 Hz tick period)	`checkTapStall`, `FixGateway.cpp:352-369`	Closes its own cluster session first, then fences and **exits**. Voluntary: rather than sit "connected" behind a dead tap, it forces the same session-loss path as the row above, which drives standby promotion on the cluster side.

The tap-stall watchdog is gated on `m_replayer.isCaughtUp()` so an in-progress cold-start/gap replay never reads as a stall, and it measures **monotonic wall-clock time**, not cluster-consensus time — deliberately, since consensus time is itself delivered by the very `Tick` frames being watched for, so it would freeze along with a stalled tap and never trip (`FixGateway.cpp:241-244`).

A `GatewayActive` naming *this* instance while it was standby is the mirror case: `m_activated` flips true and the accept gate can open once every other gate condition is met (§2.4) — no fence involved.

2.2 Standby promotion

Every logical gateway can have multiple instances (a `Gateway` row per instance, ranked by `preferenceRank`, loaded via BasicData — §5). The **cluster**, not the gateway, decides which

instance is active, by naming a `gatewayId` in a sequenced `GatewayActive` frame — every instance (and every node's Replayer-fed app) observes the same frame in the same order, so there is never a window where two instances both believe they're active off inconsistent information.

Two ways a `GatewayActive` is produced:

- **Automatic, on session close.** `Sequencer.sessionClosed` fires whenever a cluster session closes (crash, network loss, graceful shutdown — Aeron Cluster reports all of these as `onSessionClose`) and checks whether the closed session was one an *active* gateway had declared itself on via `GatewayStarted` (`Sequencer.java:174,435-445` — deliberately keyed off `GatewayStarted`, not the routing `sourceId` on every message, because other clients legitimately echo a gateway's `sourceId` and an earlier version of this logic let an unrelated `OrderExecClient` restart promote a standby out from under a perfectly healthy primary). If so, `promotionTarget` picks the lowest-`preferenceRank` sibling sharing the same `gatewaySourceId` and emits `GatewayActive` naming it — or emits nothing, fail-closed, if there is no sibling to hand over to rather than naming a nonexistent instance (`Sequencer.java:447-478`).
- **Manual, via `clusterctl activate <gatewayId>`.** Publishes an unsequenced `GatewayActive` that flows through `Sequencer.sequenceMessage` like any other ingress message — the same code path, no special-casing — and waits for its sequenced echo, matched by `gatewayId` (`ClusterCtl.java:194-291`). This is the operator's lever for a planned failover.

A **bootstrap** activation also runs once per trading day: the first `EndBasicData` (the completion marker of a reference-data load, §5) triggers `Sequencer.pendingGatewayBootstrapActivation`, which names the rank-0 (`preferenceRank == 0`) Gateway row — so exactly one instance opens its accept gate at cold start and every sibling waits as a hot standby (`Sequencer.java:409-427`). A bootstrap with no rank-0 row produces no frame — fail closed rather than guess.

2.3 Recovering FIX session state after a restart

Because `FixGateway` holds no durable state of its own, a restarting instance rebuilds what it needs from two things: an in-memory snapshot taken on the way *out* (`closeSessions`, §2.1, and the analogous `finalizeRecovery`), and lifecycle frames replayed off the cluster stream on the way back *in*.

- **On fencing/shutdown**, every active FIX session's `RecoveredSession` state — next outgoing/expected sequence numbers plus the three-field inbound-gap state (`m_inboundGapHighSeqNum`, `m_inboundGapRequestedThrough`, `m_inboundGapRunStart`, the fields `doc/gap.md`'s gap-1 fix added specifically so a mid-gap restart doesn't resume with the gap silently reopened) — is captured into `m_recoveredSessions`, keyed by client CompID.
- **On restart**, before the accept gate opens, the gateway replays `ClientConnected`/`ClientDisconnected` lifecycle frames off the cluster stream to reconstruct placeholder "recovering" connections (`FixConnection` built with `fd = -1`), then `finalizeRecovery` turns each surviving placeholder into a `RecoveredSession` entry (a crash never got to publish the matching `ClientDisconnected`, so the placeholder represents a session whose socket died with the old process) and bumps the next-connectionId counter past whatever was still pending recovery, so a freshly accepted TCP connection can never collide with one still being recovered (`FixGateway.cpp:634-664,698-734`).
- **When a new TCP connection Logons** with a CompID matching an entry in `m_recoveredSessions`, `FixConnection::adoptRecoveredSession` restores the saved sequence/gap state onto the fresh connection and consumes (erases) the entry — one-shot adoption, so the client resumes exactly where it left off rather than renegotiating from sequence 1.

2.4 The accept gate and connect-once semantics

`processSockets` only opens the accept gate when **all** of the following hold simultaneously: not already open, `m_activated` (named by a `GatewayActive`), `m_replayer.isCaughtUp()`, `m_basicDataLoaded`, and `m_ingressSender.isConnected()` (`FixGateway.cpp:384-391`) — so a gateway structurally cannot admit a FIX logon before its cluster session is live, its reference data is loaded, and it has caught up on history. `ClusterStreamSender::connect`/`connectColocated` runs exactly once, from the constructor (`FixGateway.cpp:281-282`) — there is **no in-process reconnect loop**. Once a session is lost (§2.1, row 2), the gateway exits rather than retrying; recovering service is an external supervisor's job (or a sibling instance already running as a promoted standby), not something this process attempts on its own. This is a deliberate simplification: a session-loss retry loop would have to re-derive whether it's still safe to be active, which the promotion mechanism already decides externally and unambiguously.

3. Stream recovery — cold start, gaps, and the live/history split

Every app that consumes the sequenced stream (the FIX gateway, `OrderExecClient`, `BasicDataClient`, `fix_test_server`) uses the same split: read the co-located `SequencerService`'s tap **directly and live** (untethered `aeron:ipc?tether=false`, so a slow consumer is dropped rather than backpressuring the sequencer — see §1.3 and `doc/audit.md` S4), and ask the co-located `ReplayerService` to fill in history on cold start or a detected gap.

3.1 `ReplayerService` (Java, one per node)

Off the live-delivery path entirely — no app depends on it for steady-state throughput, only for catching up. Before ever declaring itself `ready`, it replays the first frame of its own oldest tap recording and checks `globalSeqNo == 1`; if that fails, `ready` latches false for the process's lifetime (`integrityFailed`) — a deliberate refusal, because a first frame that isn't 1 means this node's own recording is missing or corrupted, and centralizing the check here means every app on the node is told `ReplayUnavailable` instead of independently discovering the same broken archive (`src/main/java/org/limitless/phixeron/replayer/ReplayerService.java`, `checkReady`/`peekFirstGlobalSeqNo`). An archive call that throws mid-replay flips the service into a `stalled` state — retried at 1s intervals, answering requests `ReplayPending` in the meantime — without crashing the process or touching live delivery, since live reads never go through this service. `MAX_CONCURRENT_REPLAYS = 2` bounds archive-IO parallelism (the only event that needs many concurrent replays is node start/restart, when every co-located app cold-starts at once); a slot freed by one client is handed to a waiting one immediately, with a 60s idle-TTL as a backstop against a client that died mid-replay. Its own control-plane replies are offered with a bounded spin and **dropped** rather than blocked past that — cheap, since the requester just resends on a timer — so one stuck app cannot couple every other app's replay to it, the same untethered-drop philosophy as the tap itself, applied to the control plane.

3.2 Client-side replay (`ReplayerStreamReceiver`, C++)

Used by `FixGateway` and the other node-local app replicas. On `start()`, it immediately requests a full walk from segment 0 (cold start). In steady state, a live-tap frame whose `globalSeqNo` jumps ahead of the next expected value clears `isCaughtUp()` and requests a **resume** at the last dispatched position — repairing just the hole rather than re-walking the whole day's log. If the resumed replay's first frame doesn't match the anchored `globalSeqNo` (meaning the active recording rotated under the client — the node it's reading from restarted), it falls back to a full re-walk.

The one subtlety worth calling out for anyone touching this code: **live-tap frames are retained, not dropped, while a walk/resume is in flight.** A frame beyond the current hole is held in a bounded pooled FIFO (`MAX_MESSAGES_FRAMES = 65536` / `MAX_MESSAGES_BYTES = 16MB`, falling

back to drop-and-rewalk past that bound) and handed over the instant the in-flight replay reaches the hole — no residual gap at the seam. The commit history is explicit about the cost of getting this wrong: dropping retained frames (as this code did until 2026-08-05) meant every walk finished one guaranteed hole short of live, and a single injected frame drop under a 300-message flood cost 6 full re-walks instead of 1 with retention. `isCaughtUp()` is a state that can re-clear on a later gap, not a one-time latch — both `FixGateway`'s tap-stall watchdog (§2.1) and leader-only emission gates (§6) depend on that, since an earlier latch-forever bug meant a mid-recovery re-walk read as a stalled sequencer and the gateway fenced itself out of a recovery it didn't need. The very first frame a client ever dispatches must carry `globalSeqNo == 1` or the process aborts — a hard invariant that this node's recording reaches the start of the log.

The replay stream is subscribed **per episode and filtered to that replay's own Aeron session id** (`openReplaySubscription`), never held open between replays. That is a flow-control requirement, not housekeeping. The `ReplayerService` answers every app on one shared `aeron:ipc` stream (201), and an Aeron publication is throttled by its slowest *tethered* subscriber — so a standing subscription there (as this code had until 2026-08-07) made every idle app a subscriber of every other app's replay, one that never polls, since `poll()` only ever reads the image of its own session. Its position sat at 0 forever, and the archive's replay wedged one publication window in and never moved again: measured on a stalled cold start, `pub-lmt` pinned at exactly 33,554,432 — 32 MiB, half a 64 MB term — with two peer `sub-pos` at 0 and the replay frozen just past it at 34.6 MB, tens of MB short of its bound. Any cold start needing more than ~32 MiB of history therefore hung **permanently**, which with no snapshots (§0) is what a restarted replica faces after a few hundred thousand messages. Filtered by session id, a replay publication has exactly one subscriber and no app can hold back another's, while the stream stays tethered so a replay fragment is still never silently dropped. This is the data-plane twin of the control-plane coupling §3.1 avoids by dropping replies rather than blocking on them; the live tap is the third case, and resolves it the other way, by being untethered (§3 above, §1.3).

Nothing detected that hang either, which was the second half of the bug. The resend timer only covers "no `Replaying` yet" (`m_awaitingReplay`), and a bounded replay of a *still-recording* segment never closes its image on reaching its bound — completion there is detected by position, not by the image closing — so an image that is attached, open, and simply frozen had no watchdog at all. A replay that makes no progress for `REPLAY_STALL_TIMEOUT_MS` (5s) now re-requests the same segment verbatim: the same recovery already used when an image closes *short* of its bound (meaning the replay was stopped under the client — superseded, slot reclaimed, archive fault), extended to cover a silent one, and a `Replaying` whose image never attaches at all. 5s is far above any legitimate pause: the archive sustains tens of MB/s into a local IPC replay, so a replay with anything left to serve is never quiet for seconds, and a spurious fire only costs one segment re-replayed.

`ClusterStreamReceiver`/`ClusterStreamClient` is the archive-direct sibling used where there's no co-located Replayer (`fix_test_server`, and `FixConnection`'s bounded resend-recovery scan): it walks an archive's recorded segments for a stream directly, replaying each historical segment fully and the last (possibly still-recording) one open-ended, so the same image delivers both historical and live messages without a subscription switch.

4. Exactly-once query replies across a failover — `OutstandingQueries`

`PortfolioQueryRequest` handling (`OrderExecClient`) needs a leader-only responder, but the leader can change mid-flight. `OutstandingQueries.hpp` (templated, Aeron-free, unit-tested standalone, mirroring the split between `Sequencer` and `SequencerService`) keeps two separate notions of state:

- **Outstanding-ness is replicated** — a request is outstanding from the moment it's sequenced until a matching *sequenced* reply discharges it (`onRequest`/`onReply`), so every

replica derives the same set of unanswered requests purely from the ordered log.

- **Dispatch is local and advisory** — "a worker on this node is handling this one" — and gets cleared by `onNotLeader()` (a leadership change: the new leader must re-consider every still-outstanding request, including ones sequenced before it ever led) or `onReplyNotEmitted()` (a reply that was attempted but never reached the cluster, e.g. a failed offer — the request stays outstanding since the log is the source of truth).

A freshly promoted leader simply calls `dispatchUndispatched()` against state every replica already holds identically — no special-cased failover recovery logic, no risk of answering a query twice (discharge only ever happens via a sequenced reply) or losing one (a reply that doesn't land leaves the request outstanding for the next leader to pick up).

Dispatch runs in **insertion order**, which — fed from the sequenced stream — is `globalSeqNo` order. That is load-bearing rather than tidiness: these side effects are externally visible in the order they are emitted (an `ExecutionReport` takes its outbound FIX `MsgSeqNum` when the gateway frames it), so iterating the underlying `unordered_map` directly, as this did until 2026-08-07, handed a counterparty a burst of acks shuffled — and shuffled *differently* per replica, since a rebuilt hash map's iteration order is not a function of the log. Determinism across replicas (§0) has to hold for emission order, not only for state.

Two more things ride the same machinery for the same reason. **Venue acks**: every `NewOrderSingle` is outstanding from the moment it is sequenced until its `ExecutionReport` is in the log, keyed on the order's `globalSeqNo` — which is also what its `ExecID` names (`VenueExecId.hpp`). That replaced a bare "am I caught up and leader?" test taken at the instant the order was decoded, with no record kept, so an order arriving before this node had processed its own promotion — or delivered by a gap-repair replay, during which `isCaughtUp()` is false throughout (§3.2) — was stepped past and acknowledged by nobody. Because the ack is a pure function of the sequenced order (cluster timestamp included), a re-emission after a failover is byte-identical to the one that may have been lost, so the at-least-once seam yields exact duplicates that dedupe on `ExecID` rather than two conflicting acks. **Rejected queries** get a second, parallel `OutstandingQueries<RejectedQuery>` instance, so the canned "queue full" reply is retried and handed off across a failover exactly like a real one instead of being fire-and-forget.

5. Reference data (BasicData) recovery

`BasicDataClient`/`Gateways` treat reference data with the same fault-tolerance shape as everything else: every node builds an identical in-memory view by following its co-located tap, so losing a node loses no reference data — a neighbour already holds it (`doc/basicdata-design.md` §0, mirroring `doc/audit.md` S1). The producer role (reading the source-of-truth and publishing `BasicData*` rows) is strictly leader-only and gated the same way as query dispatch (§4) — `isCaughtUp` plus "is this node's memberId the current leader" — so a failover doesn't produce two competing loads; the new leader's replica simply opens its own upstream connection and resumes.

Recovery deliberately has **no separate progress record**: a newly promoted leader scans what's already in the log — `EndBasicData` seen → nothing to do; all sections complete but no `EndBasicData` → emit it; otherwise resume the first incomplete section from row 0 (`doc/basicdata-design.md` §4). One rule covers every failure mode (DB error mid-section, plain crash, a failover mid-load) uniformly, because it's derived from the log rather than tracked separately from it — the same principle as §0. Consumers correspondingly commit a section only when its `remainingItems` counter reaches 0, discarding any partial scratch buffer on a restart, since this recovery rule can legitimately resend an in-flight section after a failover.

6. Operational tooling and verification

- **`clusterctl`** (`doc/clusterctl.md`) — `start`/`shutdown` bracket a run with sequenced markers so the log itself records "the cluster was up between these two points"; `activate` is the manual standby-promotion lever (§2.2); `snapshot` is explicitly refused. See §1.4.
- **Metrics** (`doc/ops.md`) — `phixeron_sequencer_tap_stalled` (latches at 1 when a node is about to terminate itself, §1.3), `phixeron_sequencer_gateway_promotion_total` (§2.2), the `phixeron_replayer_*` family (§3.1), and `phixeron_node_up` (an aggregator-synthesized per-node reachability gauge, independent of what else that node reports) give an operator the same signals this document describes, on a dashboard.
- **`src/test/scripts/chaos-runner.sh`** — randomized fault injection against a live 3-node cluster with a hot-standby gateway pair, replayable by seed. Injects: `fault_kill_leader`/`fault_kill_follower` (kill + restart a node, verifying quorum/leadership behave as above), `fault_pause_node` (`SIGSTOP` to simulate a GC pause), `fault_tap_drop` (one synthetic dropped live-tap frame, exercising §3.2's retained-message recovery), and `fault_tap_stall` (arms the same fault §1.3 self-terminates on, asserting the node actually exits rather than limping on with a dead recording). Between rounds it checks exactly one leader, a full FIX round trip against whichever gateway currently holds the accept gate, and that the live-tap consumer is still attached; at the end it freezes every member simultaneously and asserts every node's tap recording is gap-free, strictly monotone in `globalSeqNo`, and converged to the same high-water mark across all three nodes — the strongest available proof that §1.1's "byte-identical taps" invariant actually held for the run.
- **`src/test/scripts/replay-bench.sh`** — times a cold replica from launch to "Caught up" against a preloaded archive, optionally while load keeps arriving. It adds a fresh `OrderExecClient` beside a running cluster rather than restarting one (the launch script tears the cluster down when a child exits), so it walks the whole recording chain exactly as a restarted replica does. Written to chase §3.2's replay wedge and kept as the regression measurement for it: `replay-bench.sh 400000` builds ~70 MB of history — the case that used to hang forever and now converges in well under a second — and a run reporting `NEVER CAUGHT UP` is that class of bug rather than a slow machine. This is also the measurement that matters for `fault_kill_leader` above, since a restarted replica has to catch up inside the harness's probe window; the replay wedge is what made those rounds fail.

7. What this does not cover

- **No pre-trade risk gating and no matching engine** — a failover-safe query responder (§4) is not the same as a durable *order acceptance* decision; see `doc/todo.md` items 1–2.
- **No edge authentication** — CompID validation only; see `doc/todo.md` item 3. Orthogonal to recovery, but relevant if a "fault" is ever adversarial rather than accidental.
- **`aeronmd` itself and network partition/latency faults** are not exercised by `chaos-runner.sh` — noted there as unwired seams (killing the media driver is destructive to co-located C++ clients; `tc netem`/`dnctl` isn't wired up on the macOS dev host).
- **Single-node dev launches** have no failover to exercise at all — the mechanisms above only engage with 2+ cluster members.